

Assignment 8: GenAI for Software Engineering Lab

Objective

Write comprehensive test cases using PyTest for the given system.

Instructions

- Use pytest framework.
- Do NOT modify the given source code.
- Write separate test functions for each component.
- Cover edge cases, boundary cases, and negative cases.
- Use mocking where required (e.g., random.choice).
- Maintain proper naming conventions and readability.

Given Code

```
import random
import re

class ResumeParser:
    def __init__(self, resume_text):
        self.resume_text = resume_text

    def extract_skills(self):
        skills_db = ["python", "java", "sql", "machine learning", "deep learning", "nlp"]
        found_skills = []

        for skill in skills_db:
            if skill in self.resume_text.lower():
                found_skills.append(skill)

        return found_skills

    def extract_experience(self):
        match = re.search(r'(\d+)\s+years', self.resume_text.lower())
        if match:
            return int(match.group(1))
        return 0

class MCQGenerator:
    def __init__(self, skills, difficulty="easy"):
        self.skills = skills
        self.difficulty = difficulty
```

```

def generate_question(self):
    if not self.skills:
        raise ValueError("No skills provided")

    skill = random.choice(self.skills)

    if self.difficulty == "easy":
        return f"What is {skill}?"
    elif self.difficulty == "medium":
        return f"Explain key concepts of {skill}."
    elif self.difficulty == "hard":
        return f"Design a system using {skill}."
    else:
        raise ValueError("Invalid difficulty level")

class CandidateScorer:
    def __init__(self, skills, experience):
        self.skills = skills
        self.experience = experience

    def calculate_score(self):
        score = 0

        score += len(self.skills) * 10

        if self.experience > 5:
            score += 30
        elif self.experience > 2:
            score += 20
        else:
            score += 10

        return score

class InterviewEvaluator:
    def evaluate(self, answers):
        if not answers:
            raise ValueError("Answers cannot be empty")

        score = 0
        for ans in answers:
            if len(ans.strip()) > 20:
                score += 10
            else:
                score += 5

        return score

class RecruitmentSystem:
    def __init__(self, resume_text, difficulty):
        self.parser = ResumeParser(resume_text)

```

```

        self.difficulty = difficulty

    def process(self):
        skills = self.parser.extract_skills()
        experience = self.parser.extract_experience()

        scorer = CandidateScorer(skills, experience)
        score = scorer.calculate_score()

        mcq_gen = MCQGenerator(skills, self.difficulty)

        try:
            question = mcq_gen.generate_question()
        except ValueError:
            question = "No valid question generated"

        return {
            "skills": skills,
            "experience": experience,
            "score": score,
            "question": question
        }

```

Tasks

- Write test cases for ResumeParser class.
- Write test cases for MCQGenerator class.
- Write test cases for CandidateScorer class.
- Write test cases for InterviewEvaluator class.
- Write test cases for RecruitmentSystem class.

Coverage Requirements

- Functional testing
- Edge cases
- Negative testing
- Boundary testing